Laura Hietala

Oppimistehtävä 7

Tietojenkäsittelyn koulutus
TKMI23KV

14.12.2023

Sovellusohjelmointi 1
Janne Turunen

Tehtäväraportti

Tehtävänä on toteuttaa React-, MUI- ja React Router -tekniikoin sovellus, jolla lintuharrastaja voi ylläpitää tietoja lintuhavainnoinneistaan. Sovelluksen tiedot tallennetaan selaimeen localStorage-tekniikalla.

Jokainen toiminnallisuus (lisäys, muokkaus, poisto ja listaus) tulee toimia omassa näkymässään.

GitHub*:* *https://github.com/xamk-so1/oppimisteht-v-7-elahi001xamk*

Perusvaatimukset sovellukselle

Vaatimus: Sovelluksen aloitusnäkymänä on listaus merkityistä lintuhavainnoista aikajärjestyksessä (uusin ensin). Listauksessa näytetään havainnon kaikki tiedot: päivämäärä, kellonaika, havainnon paikka ja havaittu lintu.

Ratkaisu: Tämä on toteutettu BongausLista -komponentissa. Rivillä 15 luodaan sortedBongaukset-taulukko bongaukset-taulukon perusteella. Se järjestää bongaukset-taulukon aikajärjestykseen sort-funktion avulla niin, että uusimmat ovat ensin. Listaus näytetään mappaamalla sortedBongauksen havainnot ul- ja li-elementeiksi (rivit 20-36). Havainnoista luodaan linkki, joka vie yksittäisen havainnon tietoihin. Listaukseen haetaan havainnosta tiedoiksi päivämäärä, kellonaika, sijainti sekä lintulaji (riveillä 25-26).

```
15      const sortedBongaukset = bongaukset.sort((a, b) => new Date(b.pvm).getTime() - new Date(a.pvm).getTime());
16
17  return (
18      <div>
19          {sortedBongaukset.length > 0 ? (
20          <ul >
21              {sortedBongaukset.map((bongaus) => (
22                  <li key={bongaus.id} >
23
24                      <Link to={`/bongaus/${bongaus.id}`} style={{color: "#383825", textDecoration: "none"}}>
25                      {new Date(bongaus.pvm).toLocaleDateString()} Klo {new Date(bongaus.aika).toLocaleTimeString()}
26                          Paikka: {bongaus.sijainti} / Lintulaji: {bongaus.lintu}
27                      </Link>
28                      <IconButton component={Link} to={`/muokkaa/${bongaus.id}`} aria-label="Muokkaa">
29                          <EditIcon />
30                      </IconButton>
31                      <IconButton onClick={() => handleDelete(bongaus.id)} aria-label="Poista">
32                          <DeleteIcon />
33                      </IconButton>
34                  </li>
35              ))}
36          </ul>
```

Vaatimus: Sovelluksella voi lisätä uuden havainnon omassa näkymässään. Jokaisesta havainnosta kirjataan em. tiedot, mutta ainoastaan havainnon paikka ja havaittu lintu kirjoitetaan käyttäjän toimesta. Päivämäärä ja aika tulevat automaattisesti lisäyshetken perusteella. Kaikilla havainnoilla on myös yksilöivä tunniste (id), joka niin ikään luodaan automaattisesti, kun uusi havainto kirjataan.

Ratkaisu: BongausLomake-komponentissa on toiminnallisuudet uuden havainnon lisäämiseen. useState-koukussa luodaan lisays-tilamuuttuja, johon liitetään setLisays-funktio, jolla tilaa päivitetään. Kyseessä on objekti, joka pitää sisällään tiedot id (tässä kohdassa luodaan yksilöivä id automaattisesti, ja apuna käytetään uuidv4-funktiota → rivi 15), pvm, aika, sijainti ja lintu. Kokonaisuus on riveillä 14-20:

```
12  const BongausLomake: React.FC<BongausLomakeProps> = ({ bongaus, lisaaBongaus }) => {
13
14    const [lisays, setLisays] = useState<Bongaus>({
15      id: bongaus?.id || uuidv4(),
16      pvm: bongaus?.pvm || new Date().toISOString(),
17      aika: bongaus?.aika || new Date().toISOString(),
18      sijainti: bongaus?.sijainti || "",
19      lintu: bongaus?.lintu || "",
20    });
```

Päivämäärä ja aika asetetaan tätä kautta automaattisesti lisäyshetken perusteella käyttäen new Date().toISOString()-funktiota.

TextField-komponentit käsittelevät käyttäjän syötettä sijainnin ja linnun suhteen. handleChange-funktion (rivit 24-28) avulla käyttäjän syöttämät muutokset päivitetään komponentin tilaan. setLisays (rivi 27) päivittää lisays-muuttujaa kopioimalla nykyisen tilan ja korvaamalla sen tilalla, johon on viety lisätyt tiedot:

```
24  const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {
25    const { name, value } = event.target;
26
27    setLisays((prevlisays) => ({ ...prevlisays, [name]: value }));
28  };
```

Nämä tiedot lähetetään lomakkeen lähetyksessä (onSubmit, rivi 59) lisaaBongaus-funktiolle, kun käyttäjä tallentaa havainnon:

```
59      <form style={{ display: "flex", flexDirection: "column", rowGap: 20 }} onSubmit={handleSubmit}>
```

Tässä kohtaa suoritetaan handleSubmit-funktio (rivit 30-52). Siinä luodaan käyttäjän antamilla tiedoilla muokattuBongaus-objekti (rivit 33-39), joka lähetetään lisaaBongaus-funktiolle (rivi 41). lisaaBongaus-funktio on prop, joka annetaan BongausLomake-komponentille, ja sen avulla tiedot tallennetaan sovellukseen. Kun lomake on lähetetty, asetetaan uusi tyhjä tila lisays-tilalle käyttäen setLisäys-funktiota (rivit 43-49). Se nollaa lomakkeen tiedot sen jälkeen, kun käyttäjä on lähettänyt lomakkeen.

```
30  const handleSubmit = (e: React.FormEvent) => {
31    e.preventDefault();
32
33    const muokattuBongaus: Bongaus = {
34        id: lisays.id,
35        pvm: new Date(lisays.pvm).toISOString(),
36        aika: new Date(lisays.aika).toISOString(),
37        sijainti: lisays.sijainti,
38        lintu: lisays.lintu,
39      };
40
41    lisaaBongaus(muokattuBongaus);
42
43    setLisays({
44      id: uuidv4(),
45      pvm: "",
46      aika: "",
47      sijainti: "",
48      lintu: "",
49    });
50
51    navigate("/");
52  };
```

Riveillä 61-76 on TextField-komponentit (kuva alla), joissa käyttäjä voi syöttää sijainnin sekä linnun. Kenttiin liitetty handleSubmit (rivi 59) käsittelee kuitenkin myös päivämäärän ja ajan, jotka tulevat automaattisesti sitä kautta, vaikka niitä ei erikseen kirjata (new Date, luo uuden Date-objektin joka edustaa nyky-hetkeä, ks. ed. kuva rivit 35 ja 36).

```
59      <form style={{ display: "flex", flexDirection: "column", rowGap: 20 }} onSubmit={handleSubmit}>
60
61        <TextField
62          label="Paikka"
63          type="text"
64          name="sijainti"
65          value={lisays.sijainti}
66          onChange={handleChange}
67          required
68        />
69        <TextField
70          label="Lintu"
71          type="text"
72          name="lintu"
73          value={lisays.lintu}
74          onChange={handleChange}
75          required
76        />
77
```

Vaatimus: Yksittäinen havainto voidaan poistaa listauksesta. Poisto voidaan aloittaa listauksen yhteydessä näkyvää ikonia tai painiketta painamalla, mutta varsinainen poistaminen vahvistetaan omassa näkymässään.

Ratkaisu: BongausLista-komponentissa on jokaisen havainnon kohdalla kaksi painiketta (ikonit). Toinen on muokkausta varten ja toinen poistamista. Ikonit määritellään havaintolistaan riveillä 28-33.

```
15      const sortedBongaukset = bongaukset.sort((a, b) => new Date(b.pvm).getTime() - new Date(a.pvm).getTime());
16
17    return (
18        <div>
19            {sortedBongaukset.length > 0 ? (
20            <ul >
21                {sortedBongaukset.map((bongaus) => (
22                    <li key={bongaus.id} >
23
24                        <Link to={`/bongaus/${bongaus.id}`} style={{color: "#383825", textDecoration: "none"}}>
25                        {new Date(bongaus.pvm).toLocaleDateString()} Klo {new Date(bongaus.aika).toLocaleTimeString()}:
26                            Paikka: {bongaus.sijainti} / Lintulaji: {bongaus.lintu}
27                        </Link>
28                        <IconButton component={Link} to={`/muokkaa/${bongaus.id}`} aria-label="Muokkaa">
29                            <EditIcon />
30                    </IconButton>
31                        <IconButton onClick={() => handleDelete(bongaus.id)} aria-label="Poista">
32                            <DeleteIcon />
33                        </IconButton>
34                    </li>
```

Muokkausnappi vie Bongauksen muokkaus -näkymään (reititys rivillä 28), jossa vielä esitellään muokattava havainto ja jossa on myös nappi sitä varten,

että voi palata listaukseen.



Muokkaa havaintoa-napista taas pääsee varsinaiselle muokkauslomakkeelle. Poistonappi käynnistää handleDelete-funktion (rivit 10-14 alla ja ed. sivulla rivi 31), joka laukaisee viesti-ikkunan (rivi 11, alla), jossa poistaminen pitää vielä vahvistaa.

```
10      const handleDelete = (id : string) => {
11          if (window.confirm("Haluatko varmasti poistaa tämän havainnon?")) {
12              poistaBongaus(id);
13          }
14      };
```



App-komponentissa poistaBongaus-funktio (rivit 23-27) päivittää bongaukset-tilaa poistamalla havainnon sen id:n perusteella. Lisäksi se päivittää localStoragea.

```
23  const poistaBongaus = (id : string) => {
24    const paivitetytBongaukset = bongaukset.filter((bongaus) => bongaus.id !== id);
25    setBongaukset(paivitetytBongaukset);
26    localStorage.setItem("bongaukset", JSON.stringify(paivitetytBongaukset));
27  };
```

Vaatimus: Havainnointitiedot tallentuvat käyttäjän selaimeen localStorage-tekniikoilla aina, kun uusi tieto lisätään tai poistetaan. Vastaavasti localStorageen tallennetut tiedot luetaan sovellukseen aina, kun sovellus käynnistetään uudestaan.

Ratkaisu: Rivillä 12 käytetään useState-koukkua määrittämään bongaukset-tila sekä setBongaukset-funktio sitä päivittämään. Rivillä 13 alustetaan tämä tila localStoragesta tai vaihtoehtoisesti tyhjällä taulukolla.
Funktiot lisaaBongaus (rivit 16-21) ja poistaBongaus (rivit 23-27) päivittävät bongaukset-tilaa ja tallentavat päivitykset localStorageen (rivit 20 ja 26). useEffect-koukku riveillä 29-34 lukee tietoja localStoragesta (rivi 30) ja vie ne bongaukset-tilaan (rivi 32). useEffect riveillä 36-38 vie päivitetyt bongaukset localStorageen aina, kun bongaukset päivittyy.

```
11  const App : React.FC = () : React.ReactElement => {
12    const [bongaukset, setBongaukset] = useState<Bongaus[]>(
13      JSON.parse(localStorage.getItem("bongaukset") || "[]")
14    );
15
16    const lisaaBongaus = (bongaus : Bongaus) => {
17      const uusiBongaus = { ...bongaus, id: uuidv4() };
18      const paivitetytBongaukset = [...bongaukset, uusiBongaus];
19      setBongaukset(paivitetytBongaukset);
20      localStorage.setItem("bongaukset", JSON.stringify(paivitetytBongaukset));
21    };
22
23    const poistaBongaus = (id : string) => {
24      const paivitetytBongaukset = bongaukset.filter((bongaus) => bongaus.id !== id);
25      setBongaukset(paivitetytBongaukset);
26      localStorage.setItem("bongaukset", JSON.stringify(paivitetytBongaukset));
27    };
28
29    useEffect(() => {
30      const storedData = localStorage.getItem("bongaukset");
31      if (storedData) {
32        setBongaukset(JSON.parse(storedData));
33      }
34    }, []);
35
36    useEffect(() => {
37      localStorage.setItem("bongaukset", JSON.stringify(bongaukset));
38    }, [bongaukset]);
```

Lisävaatimukset sovellukselle

Vaatimus: Yksittäisen havainnon tietoja voi muokata omassa näkymässään. Muokkaustilaan pääsee omalla ikonillaan/painikkeellaan vastaavalla tavalla kuin poistaessa. Muokkaustilassa käyttäjä voi muokata myös päivämäärää ja kellonaikaa. Luonnollisesti muutokset tallennetaan myös localStorageen.

Ratkaisu: Muokkaustilaan liittyvä painike esiteltiin jo aiemmin tässä raportissa sivulla 4. Siellä myös esiteltiin, miten yksittäisen havainnon tietoja pääsee muokkaamaan, nämä ikonit on määritelty BongausLista-komponentissa. Muokkaus-ikonilla siis pääsee näkymään, jossa bongausta voi muokata. Siellä on Muokkaa havaintoa -nappi. Se on määritelty MuokkaaLomaketta-komponentissa riveillä 56-58:

```
56          <Button variant="contained"
57          style={{ backgroundColor: "#ffffe6", color: "#78774f" }}
58          onClick={() => setMuokkaustila2(true)}>Muokkaa havaintoa</Button>
59          <Link to="/">
60          <Button variant="contained" style={{ backgroundColor: "#ffffe6", color:
61          Takaisin listaukseen
62        </Button>
63        </Link>
64          </Box>
65
66        ) : (
67          <BongausLomake bongaus={tilaBongaus} lisaaBongaus={muokkaaBongausta} />
68        )}
```

Kun käyttäjä klikkaa sitä, muokkaustila2-muuttuja asetetaan trueksi, jolloin BongausLomake renderöidään (rivi 67) ja käyttäjä voi muokata havainnon tietoja muokkauslomakkeen TextFieldeihin (rivit 61-76) sekä DateTimePickeriin (rivit 81-86) ja TimePickeriin (rivit 88-93). Näiden Pickereiden avulla voidaan muokata päivämäärää ja kellonaikaa. TextFieldien onChange-tapahtumankäsittelijät (rivit 66 ja 74) käynnistävät handleChange-funktion suorittamisen (esitelty sivulla 3), jonka avulla käyttäjän syöttämät muutokset päivitetään komponentin tilaan. Pickereiden onChange-tapahtumankäsittelijät riveillä 84 ja 91 käynnistää setLisays-funktion, joka päivittää lisays-tilamuuttujaa (esitelty sivulla 2). Lopputuloksena kentän arvo päivitetään uudella arvolla (eli käyttäjän muokkaamalla arvolla).

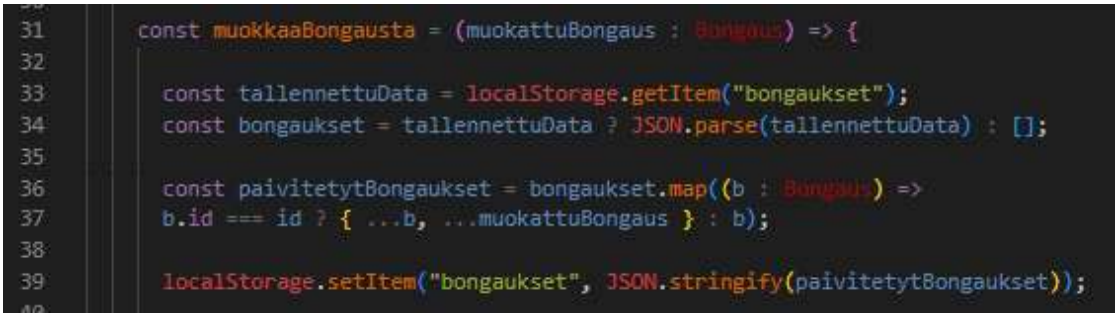
```
61          <TextField
62            label="Paikka"
63            type="text"
64            name="sijainti"
65            value={lisays.sijainti}
66            onChange={handleChange}
67            required
68          />
69          <TextField
70            label="Lintu"
71            type="text"
72            name="lintu"
73            value={lisays.lintu}
74            onChange={handleChange}
75            required
76          />
77
78          {bongaus && (
79            <>
80              <LocalizationProvider dateAdapter={AdapterDateFns} adapterLocale={fi}>
81              <DateTimePicker
82                label="Bongauksen päivämäärä"
83                value={new Date(lisays.pvm)}
84                onChange={(newValue) => setLisays((prev : any) =>({ ...prev, pvm: newValue || new Date() }))}
85                views={["day", "month", "year"]}
86              />
87
88              <TimePicker
89                label="Bongauksen kellonaika"
90                value={new Date(lisays.aika)}
91                onChange={(newValue) => setLisays((prev : any) => ({ ...prev, aika: newValue || new Date() }))}
92                views={["hours", "minutes"]}
93            />
94              </LocalizationProvider>
95            </>
96          )}
97          <Button type="submit" variant="contained" style={{backgroundColor: "#ffffe6", color: "#78774f", width: "100%"}}>
98            {bongaus ? "Tallenna muutokset" : "Lisää tämä lintunen"}
99          </Button>
100         <Link to="/">
101         <Button variant="contained" style={{ backgroundColor: "#ffffe6", color: "#78774f", width: "100%" }}>
102           Takaisin listaukseen
103         </Button>
104       </Link>
105       </form>
106     </Box>
107   );
108 };
109
```

muokkaaBongausta-funktio (rivit 31-43) tallentaa muutokset localStorageen. Se ottaa parametrina muokattuBongaus -objektin (rivi 31), joka sisältää muokatut tiedot havainnosta. Rivillä 33 const tallennettuDataan haetaan ja tallennetaan tallennetut havainnot localStoragesta.

Rivin 34 const bongaukset jne tarkistaa, onko tallennettua dataa olemassa ja jos on, parsii sen JSON-muotoon.

Riveillä 36-37 mappaus käy läpi kaikki tallennetut bongaukset ja päivittää sen, jonka id vastaa muokattavan havainnon id:tä. Spredaamalla kopioidaan kaikki alkuperäisen havainnon ominaisuudet ja sitten päivitetään ne muokattuBongaus-objektin mukaisesti.

Rivillä 39 tallennetaan päivitetyt havainnot takaisin localStorageen JSON-muodossa.



```
31    const muokkaaBongausta = (muokattuBongaus : Bongaus) => {
32
33      const tallennettuData = localStorage.getItem("bongaukset");
34      const bongaukset = tallennettuData ? JSON.parse(tallennettuData) : [];
35
36      const paivitetytBongaukset = bongaukset.map((b : Bongaus) =>
37      b.id === id ? { ...b, ...muokattuBongaus } : b);
38
39      localStorage.setItem("bongaukset", JSON.stringify(paivitetytBongaukset));
```